

The background features a light gray grid with faint geometric lines. Scattered across the grid are several 3D rectangular blocks in orange, blue, green, and gray, some of which are interconnected. A white rectangular window with a light blue header and three small gray circles in the top-left corner is centered on the slide. Inside the window, the title and subtitle are displayed in a clean, sans-serif font. Below the subtitle, a light blue rounded rectangle contains the phrase 'Dominando la sintaxis del diseño de sistemas.'

Ingeniería de Requerimientos: De la Ambigüedad a la Precisión

El conjunto de herramientas visuales para escribir
requerimientos con PLANguage y EARS

Dominando la sintaxis del diseño de sistemas.

El Problema del Lenguaje Natural

Los autores sin entrenamiento formal escriben requerimientos usando lenguaje natural sin restricciones. Esto genera un efecto dominó de defectos en el software.



Lenguaje Natural Sin Restricciones

Ambigüedad y Subjetividad

Palabras como '**rápido**' o '**soportar**' tienen múltiples interpretaciones.

Omisión

Lógica Incompleta. Se define el '**qué pasa si**', pero se olvida el '**y si no**'.

Incomprobabilidad

No Testeable. Si no se puede **medir**, el equipo de validación no puede **probarlo**.

La Anatomía de la Calidad (Criterios de Excelencia)

3-Pillar Quality Matrix

Contenido (El Qué)

- ✓ **Completo:** Sin brechas que obliguen al desarrollador a adivinar.
- ✓ **Correcto:** Refleja la necesidad real.
- ✓ **Necesario:** Aporta valor real al sistema.

Lenguaje (El Cómo)

- ✓ **Conciso:** Directo, sin verbosidad innecesaria.
- ✓ **No Ambiguo:** Posee una única interpretación posible.
- ✓ **Consistente:** No contradice otros requerimientos.

Gestión (El Control)

- ✓ **Priorizado:** Clasificado por importancia o valor.
- ✓ **Verificable:** Se puede probar o medir de forma objetiva.
- ✓ **Trazable:** Posee un identificador único.

La Macro-Estructura: PLANguage

Desarrollado por Tom Gilb, PLANguage es un lenguaje natural restringido. Envuelve la redacción del requerimiento con metadatos cruciales para su gestión.

Ticket #104: Envío y Facturación

Nombre:

I. Send, Shipping

Prioridad:

Alta (Razón: Impacto directo en ingresos)

Estado:

Comprometido

Justificación (Rationale):

Necesario para procesar órdenes no prepagadas.

Requerimiento (Texto EARS):

[Bloque reservado para la sintaxis exacta]

Trazabilidad: Creado por Julie English | Versión 1.1

La Macro-Estructura: PLANguage

Name	A short, descriptive name
Requirement	Text that defines the requirement, following EARS*
Rationale	The reasoning that justifies the requirement
Priority	Sets the importance of the requirement relative to others in the product
Priority Reason	A brief justification for the assigned priority level
Status	The current state of the requirement
Contact	The person to contact with questions about the requirement
Source	The original inspiration for the requirement

Planguage Example

Name: Create_Invoice

Requirement: When an Order is shipped and Order Terms are not “Prepaid”, the system shall create an Invoice.

Rationale: Task automation decreases error rate, reduces effort per order.
Meets corporate business principle for accounts receivable.

Priority: High.

Priority Reason: If not implemented, business process reengineering will be necessary and program ROI will drop by \$400K per year. ← Finance study

Status: Committed

Contact: Hugh P. Essen

Source: I. Send, Shipping

Created by: Julie English

Version: 1.1, **Modified Date:** 20 Oct 11

La Micro-Sintaxis: Introducción a EARS

EARS (Easy Approach to Requirements Syntax) fue creado por Alistair Mavin en Rolls-Royce y adoptado rápidamente por Intel debido a su poder y simplicidad para eliminar defectos en equipos globales.



La Fórmula Genérica: Construyendo con Bloques

Los requerimientos funcionales en EARS no se escriben como literatura; se ensamblan.



Quando se envía
un pedido,

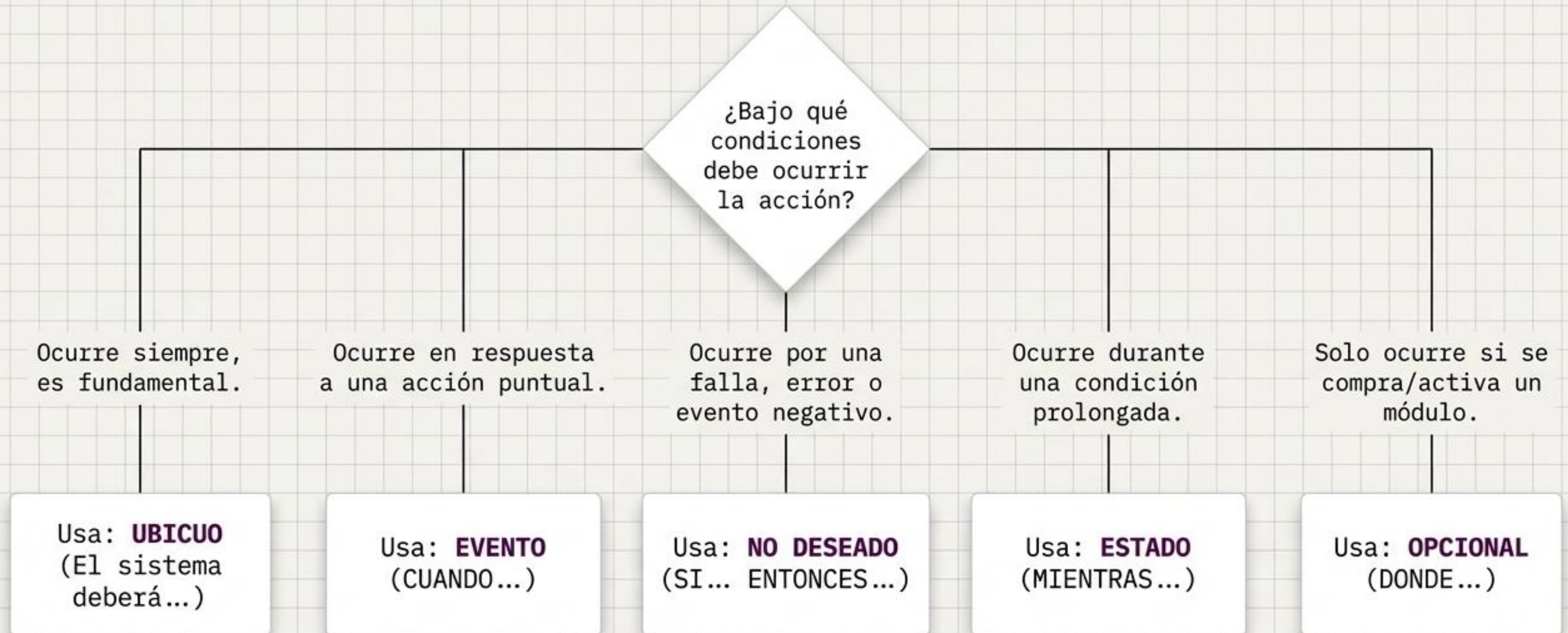
el sistema

deberá crear

una factura.

El Árbol de Decisión EARS

¿Cómo saber qué patrón utilizar? Hazte la siguiente pregunta antes de escribir:



El Framework EARS: Matriz de Síntesis



Ubicuos

(Sin palabra clave)

Propiedad universal permanente.



Eventos

CUANDO

Ocurre ante un estímulo.



No Deseado

SI / ENTONCES

Manejo de fallos y errores.



Estado

MIENTRAS

Depende de un estado operativo.



Opcional

DONDE

Depende de una característica instalada.

Nota Analítica: Existe un sexto nivel (Complejo) que combina lógicamente dos o más de estos patrones base.

Patrón 1: Requisitos Ubicuos

Definición: Definen una propiedad fundamental del sistema. Existen en todo momento. No tienen precondiciones ni disparadores.

`<nombre del sistema>` deberá `<respuesta del sistema>`

El paquete de software deberá incluir un instalador.

El software deberá estar escrito en Java.

El software deberá estar disponible para su compra en el sitio web de la empresa y en tiendas minoristas.

Patrón 2: Dirigidos por Eventos (Event-Driven) ⚡

Definición: Se inician única y exclusivamente cuando ocurre un evento o disparador específico.

CUANDO <disparador>, el <nombre del sistema> deberá <respuesta del sistema>

CUANDO un Dispositivo No Registrado se conecte a un puerto USB, el **SO** deberá intentar localizar y cargar el controlador.

CUANDO se inserte un DVD en el reproductor, el **SO** deberá hacer girar la unidad óptica.

CUANDO el nivel de agua caiga por debajo del Umbral Bajo, el **software** deberá abrir la válvula para llenar el tanque al Umbral Alto.

Patrón 3: Comportamientos No Deseados (*Unwanted Behavior*) ⚠

Definición: Maneja errores, fallos, perturbaciones y otros eventos indeseados.

SI *<condición indeseada>*, **ENTONCES** el *<nombre del sistema>* deberá *<respuesta del sistema>*

SI *las velocidades medida y calculada varían en más de un 10%*, **ENTONCES** el **software** deberá *usar la velocidad medida*.

SI *la suma de comprobación (checksum) de memoria es inválida*, **ENTONCES** el **software** deberá *mostrar un mensaje de error*.

SI *la tarjeta de cajero insertada se reporta como perdida o robada*, **ENTONCES** el **software** deberá *deberá confiscar la tarjeta*.

Patrón 4: Dirigidos por Estado (State-Driven)

Definición: Se desencadenan o aplican mientras el sistema se encuentra en un estado operativo específico sostenido en el tiempo.

MIENTRAS *<estado del sistema>*, el *<nombre del sistema>* deberá *<respuesta del sistema>*

MIENTRAS *esté en Modo de Bajo Consumo*, el **software** deberá **mantener el brillo de la pantalla en el Nivel Mínimo.**

MIENTRAS *el calentador esté encendido*, el **software** deberá **cerrar la válvula de entrada de agua.**

MIENTRAS *el piloto automático esté activado*, el **software** deberá **mostrar una indicación visual al piloto.**

Patrón 5: Características Opcionales (*Optional Feature*)

Definición: Se invocan solo en sistemas que incluyen una característica opcional o hardware particular.

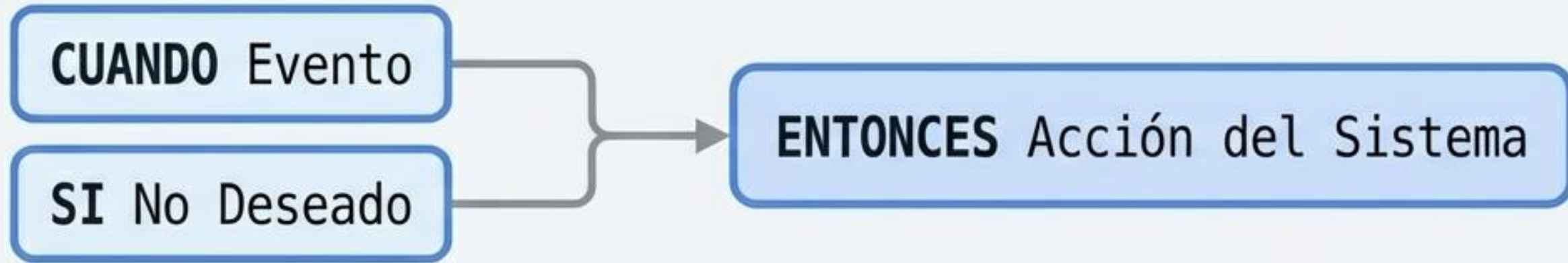
DONDE *<se incluya la característica>*, el *<nombre del sistema>* deberá *<respuesta del sistema>*

DONDE *un tesauro sea parte del paquete de software*, el **instalador** deberá **consultar al usuario** antes de instalarlo.

DONDE *haya cifrado por hardware instalado*, el **software** deberá **cifrar los datos** usando el hardware en lugar de un algoritmo de software.

DONDE *un puerto HDMI esté presente*, el **software** deberá **permitir al usuario seleccionar contenido HD** para visualización.

Patrón 6: Requisitos Complejos (Combinaciones)



Definición: Describen eventos condicionales complejos que involucren múltiples disparadores, estados y/o características opcionales combinando palabras clave.

Evento + No Deseado

CUANDO se presiona el botón del tren de aterrizaje,
SI el software detecta que el tren no se bloquea en su posición,
ENTONCES el software **deberá emitir una alarma.**

Opcional + Evento

DONDE haya una segunda unidad óptica instalada,
CUANDO el usuario seleccione copiar discos,
el software **deberá mostrar una opción para copiar directamente de una unidad a otra.**

Laboratorio de Reescritura 1: Detectando Disparadores Faltantes



Antes (Original)

El software deberá
llamar a la Compañía
de Alarmas.

Falso requisito ubicuo. ¿Llama a la
compañía todo el tiempo? Falla la
lógica por omisión del disparador real.

Patrón Aplicado:
Comportamiento
No Deseado

Después (EARS)



SI el software de
alarma *detecta que un
sensor ha fallado*,
ENTONCES el software
de alarma deberá
llamar a la **Compañía
de Alarmas para
reportar el fallo.**

Laboratorio de Reescritura 2: Definiendo Estados Ocultos



Antes (Original)

El software deberá
silenciar el
micrófono.

¿Es una acción única? Si es así,
¿qué la revierte? Oculta una
dependencia de estado continuo.

Patrón Aplicado:
Dirigido por
Estado

Después (EARS)



MIENTRAS el *botón de
silencio esté
presionado*, el
software deberá
**silenciar el
micrófono.**

Laboratorio de Reescritura 3: Aclarando Dependencias



Antes (Original)

El software deberá
descargar el libro
sin costo.

Patrón Aplicado:
Característica
Opcional

¿Aplica para absolutamente todos los
libros del catálogo? Falta declarar la
precondición o característica requerida.

Después (EARS)



DONDE *el libro esté
disponible en formato
digital*, el **software**
deberá permitir al
usuario
**descargar el libro
sin costo.**

La Matriz de Referencia EARS (Cheat Sheet)

Tipo de Patrón	Palabra Clave	Fórmula Estructural	Cuándo Usarlo
Ubicuo	(Ninguna)	Sistema + Respuesta	Propiedades fundamentales y constantes.
Impulsado por Evento	CUANDO	CUANDO [evento], Sistema + Respuesta	Reacciones a estímulos específicos y puntuales.
Impulsado por Estado	MIENTRAS	MIENTRAS [estado], Sistema + Respuesta	Acciones atadas a un modo o condición persistente.
Comportamiento No Deseado	SI / ENTONCES	SI [fallo], ENTONCES Sistema + Respuesta	Manejo de excepciones, fallos y seguridad.
Característica Opcional	DONDE	DONDE [módulo], Sistema + Respuesta	Variantes de producto o características configurables.

Reflexiones Finales

EARS y **PLANguage** no son solo reglas gramaticales; son herramientas de diagnóstico.

Obligan a los autores a identificar condiciones ocultas, eliminar la ambigüedad y separar las reglas de negocio del comportamiento del sistema.

Benefician tanto a los desarrolladores que construyen el sistema como a los equipos de validación que lo prueban.

PLANguage

“La precisión previene las suposiciones. Diseña tus requerimientos con la misma rigurosidad con la que diseñarías tu código.”

Tipo de Patrón	Palabra Clave	Fórmula Estructural	Ejemplo
Comportamiento No Deseado	SI / ENTONCES	SI [fallo], ENTONCES Sistema + Respuesta	Propiedad constante
Característica Opcional	DONDE	DONDE [módulo], Sistema + Respuesta	Reacción específica
			Acciones condicionales
			Manejo de errores y seguridad
			Variantes de características